EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

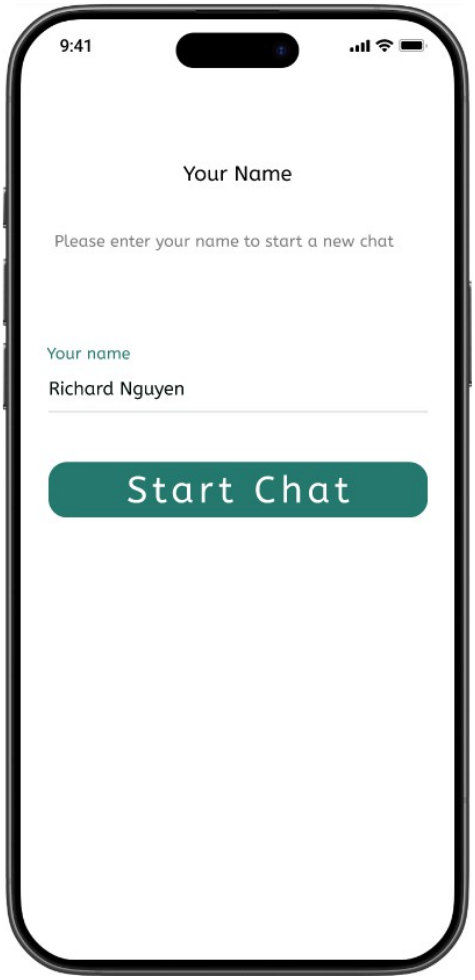
Practice Assignment – Quarter 1, 2026-2027
Course Name: Advanced Mobile Programming
Course Code: CSW 434
Student's Full Name:
Student ID:

PracticeAssignment1

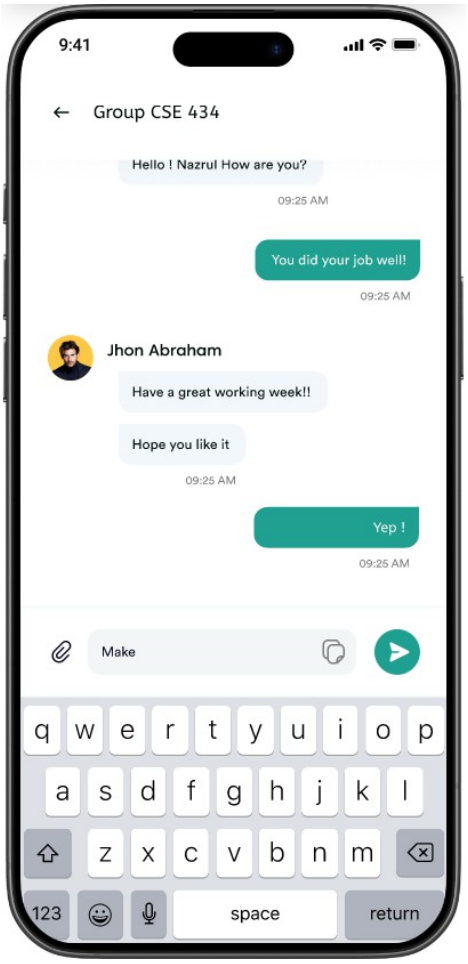
Group Chat App with Real-Time Feature using Socket

Objective:

- Build a basic mobile application using React Native.
- Get familiar with essential UI components such as TextInput, Button, and FlatList.
- Implement real-time messaging using Socket.IO as a supplementary feature to enhance the app.



Screen 1



Screen 2

Assignment Requirements

1. Main Functionality:

Develop a mobile app that displays a simple chat interface with the following Screen:

Screen 1 :

- Home Screen (Screen 1), this is the entry screen where users can enter their name before starting the chat.

Flow:

1. The user inputs their name in the `TextInput`.
2. Upon clicking the "Start Chat" button, the app stores the user's name locally and navigates to the **Group Screen**.
3. The user's name will be sent to the server via a socket event to notify that a new user has joined the group.

Screen 2 :

- Group Screen (Screen 2), this screen displays the real-time group chat interface where multiple users can exchange messages

Flow:

1. When the user enters a message and clicks "Send", the message is sent to the server via a socket event.
2. The server broadcasts the message to all connected users.
3. The app receives the broadcasted message and updates the chat interface, displaying the new message in the list.

2. Real-Time Feature

Once we have completed the interface with two screens, our application acts as the **client**. The next step is to build a **Socket.IO server** to manage and distribute real-time messages in the mobile application.

Main Server Functions:

1. **Connecting with the Client**

   The server will use **Socket.IO** to create a real-time connection between the client (React Native app) and the server.

2. **Real-time Message Broadcasting**

   When a user sends a message, the server will receive it and **broadcast** it to all connected clients. This ensures that messages are instantly synchronized across all devices.

3. **Supporting Multiple Users Simultaneously**

   The server must support simultaneous connections from multiple users and handle messages within the same group chat.

Server Environment:

- **During Development**:

The server can run **locally** on the same machine as the client. For example, you can run the server on your computer and connect to it from the React Native app running on a physical device or an emulator.

- **For Testing or Deployment (Optional)**:
  The server can be deployed on a **remote virtual machine** or a cloud service such as **Heroku**, **DigitalOcean**, or **AWS**, allowing it to be accessible from any device.

Key Requirements for Real-Time Features:

- Messages from one user must be broadcast to **all other connected users** instantly.
- The server must handle **multiple simultaneous connections** and ensure minimal latency when transmitting messages.